ISE 291 Final Project Report: Telco Customer Churn Prediction

Group Members and Contributions

Name	ID	Contribution
Majed AlOwayyeidh	202467820	Integrated all milestones (M1–M3), organized the final report, and coordinated the modeling workflow
Talal AlDakheel	202467400	Supported model evaluation and ensured consistency between business objectives and final analysis results
Ahmed AlSaad	202432980	Implemented preprocessing pipeline and contributed to model building (Logistic Regression, Decision Tree, I
Abdullah Alluqmani	202257720	Interpreted model results in relation to EDA findings and contributed to feature-level analysis
Rakan AlShalan	202351430	Reviewed final report structure, ensured compliance with rubric requirements, and prepared the notebook fo

Project Objective

The objective of this project is to analyze customer churn behavior in a telecom company and build classification models that can predict whether a customer is likely to churn.

1. Introduction
2. Dataset Description
3. Data Preparation
4. Exploratory Data Analysis
5. Model Building
6. Model Evaluation
7. Model Comparison
8. Final Model Selection
9. Interpretation of Results
10. Conclusion
11. Operationalization

Import libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_ma

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

import warnings
warnings.filterwarnings("ignore")
```

Upload dataset

```
from google.colab import files
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files telco_churn_cleaned.xlsx

telco_churn_cleaned.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 642260 bytes, last modified: 3/13/2026 - 100% done

```
Saving telco_churn_cleaned.xlsx to telco_churn_cleaned (1).xlsx
```

Read dataset

```
df = pd.read_excel("telco_churn_cleaned.xlsx")

df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	

5 rows × 21 columns

Basic dataset info

```
print("Shape:", df.shape)
display(df.info())
display(df.describe(include="all").T)
```

```
Shape: (7043, 21)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   float64
 20  Churn             7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB
None
```

Dataset Description

The dataset contains customer-level telecom information. Each row represents one customer, and the target variable is customer churn. The dataset includes both numerical and categorical variables, which makes it suitable for classification analysis.

```
df.columns
```

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

Target Variable

The target variable is `Churn`, which indicates whether a customer left the company or not. Since the target has two possible outcomes, this is a binary classification problem.

Check missing values

```
df.isnull().sum()
```

	count	unique	top	freq	mean	std	min	25%	50%	
customerID	7043	7043	3186-AJGDW	1	NaN	NaN	NaN	NaN	NaN	N
gender	7043	2	Male	3555	NaN	NaN	NaN	NaN	NaN	N
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612	0.0	0.0	0.0	
Partner	7043	2	No	3641	NaN	NaN	NaN	NaN	NaN	N
Dependents	7043	2	No	4933	NaN	NaN	NaN	NaN	NaN	N
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481	0.0	9.0	29.0	5
PhoneService	7043	2	Yes	6361	NaN	NaN	NaN	NaN	NaN	N
MultipleLines	7043	3	No	3390	NaN	NaN	NaN	NaN	NaN	N
InternetService	7043	3	Fiber optic	3096	NaN	NaN	NaN	NaN	NaN	N
OnlineSecurity	7043	3	No	3498	NaN	NaN	NaN	NaN	NaN	N
OnlineBackup	7043	3	No	3088	NaN	NaN	NaN	NaN	NaN	N
DeviceProtection	7043	3	No	3095	NaN	NaN	NaN	NaN	NaN	N
TechSupport	7043	3	No	3473	NaN	NaN	NaN	NaN	NaN	N
StreamingTV	7043	3	No	2810	NaN	NaN	NaN	NaN	NaN	N
StreamingMovies	7043	3	No	2785	NaN	NaN	NaN	NaN	NaN	N
Contract	7043	3	Month-to-month	3875	NaN	NaN	NaN	NaN	NaN	N
PaperlessBilling	7043	2	Yes	4171	NaN	NaN	NaN	NaN	NaN	N
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN	NaN	NaN	NaN	N
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047	18.25	35.5	70.35	89
TotalCharges	7043.0	NaN	NaN	NaN	2279.734304	2266.79447	0.0	398.55	1394.55	378
Churn	7043	2	No	5174	NaN	NaN	NaN	NaN	NaN	N

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

Check churn distribution

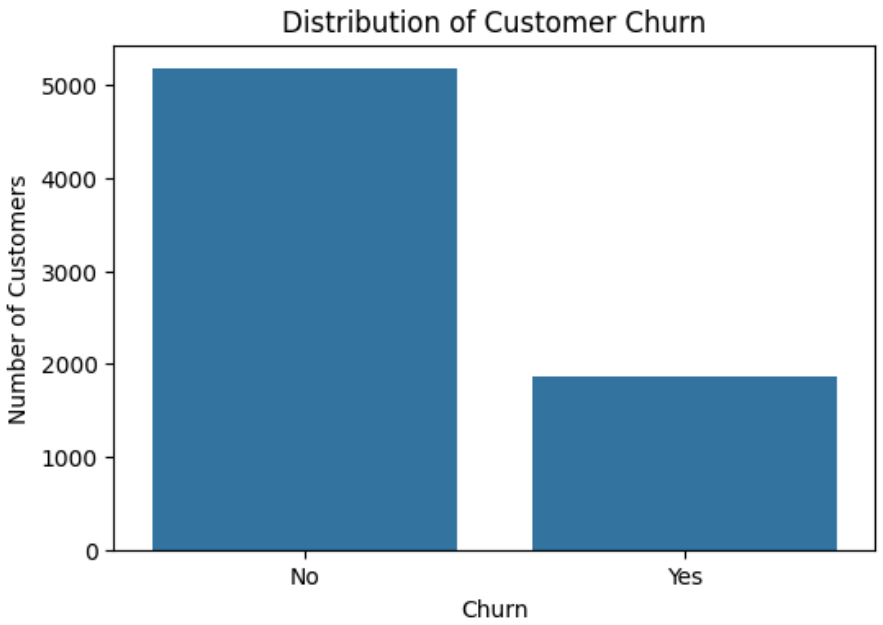
```
df["Churn"].value_counts()
```

Churn	count
No	5174
Yes	1869

dtype: int64

Plot churn distribution

```
plt.figure(figsize=(6,4))
sns.countplot(data=df, x="Churn")
plt.title("Distribution of Customer Churn")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
plt.show()
```



Churn Distribution

The churn distribution shows how many customers stayed and how many customers left. This is important because classification models can be affected if one class is much larger than the other.

Prepare data for modeling

```
# Drop customerID because it is only an identifier and does not help prediction
df_model = df.drop(columns=["customerID"])

# Separate features and target
X = df_model.drop(columns=["Churn"])
y = df_model["Churn"]

# Convert target to binary values
y = y.map({"No": 0, "Yes": 1})

# Identify categorical and numerical columns
categorical_cols = X.select_dtypes(include=["object"]).columns
numerical_cols = X.select_dtypes(include=["int64", "float64"]).columns

print("Categorical columns:", list(categorical_cols))
print("Numerical columns:", list(numerical_cols))
```

```
Categorical columns: ['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'Interne
Numerical columns: ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (5634, 19)
Test shape: (1409, 19)
```

Preprocessing pipeline

```
# Scale numerical features and encode categorical features
#preprocessor = ColumnTransformer(
#    transformers=[
#        ("num", StandardScaler(), numerical_cols),
#        ("cat", pd.get_dummies, categorical_cols)  # temporary (will fix below)
#    ]
#)
```

```
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
    ]
)
```

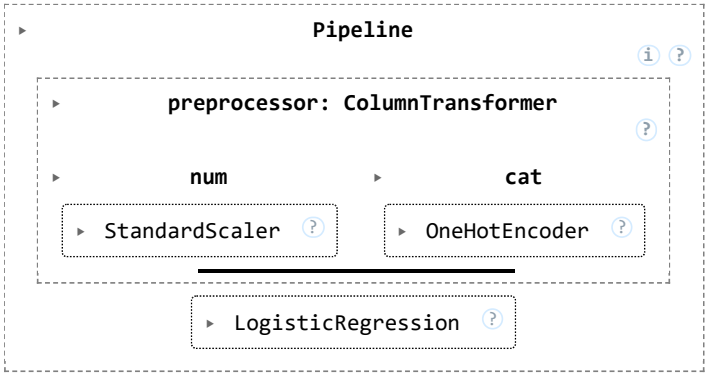
Model Building

In this section, multiple classification models are trained and evaluated. The goal is to compare their performance and select the best model for predicting customer churn.

First Model: Logistic Regression

```
log_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

log_model.fit(X_train, y_train)
```



Predictions

```
y_pred_log = log_model.predict(X_test)
```

Evaluation

```
print("Logistic Regression Performance:")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Precision:", precision_score(y_test, y_pred_log))
print("Recall:", recall_score(y_test, y_pred_log))
print("F1 Score:", f1_score(y_test, y_pred_log))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_log))
```

```
Logistic Regression Performance:
Accuracy: 0.8055358410220014
Precision: 0.6572327044025157
Recall: 0.5588235294117647
F1 Score: 0.6040462427745664

Confusion Matrix:
[[926 109]
 [165 209]]
```

Double-click (or enter) to edit

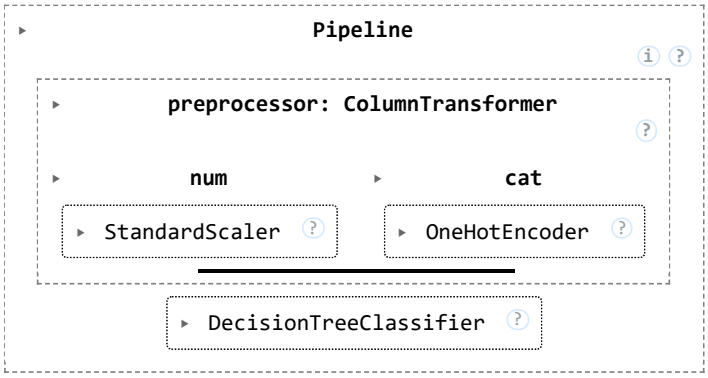
Logistic Regression Interpretation

Logistic regression is used as a baseline model. It helps in understanding how different features contribute to customer churn and provides a simple but effective benchmark for comparison.

Decision Tree Model

```
tree_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(random_state=42))
])

tree_model.fit(X_train, y_train)
```



Predictions

```
y_pred_tree = tree_model.predict(X_test)
```

Evaluation

```
print("Decision Tree Performance:")
print("Accuracy:", accuracy_score(y_test, y_pred_tree))
print("Precision:", precision_score(y_test, y_pred_tree))
print("Recall:", recall_score(y_test, y_pred_tree))
```

```
print("F1 Score:", f1_score(y_test, y_pred_tree))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_tree))
```

```
Decision Tree Performance:
Accuracy: 0.7210787792760823
Precision: 0.4754521963824289
Recall: 0.4919786096256685
F1 Score: 0.4835742444152431

Confusion Matrix:
[[832 203]
 [190 184]]
```

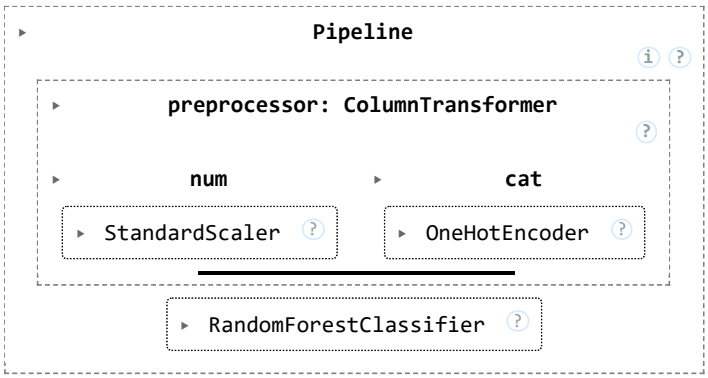
Decision Tree Interpretation

The decision tree model captures non-linear relationships between features and churn. However, it may overfit the training data, which can affect its performance on unseen data.

Random Forest Model

```
rf_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(n_estimators=100, random_state=42))
])

rf_model.fit(X_train, y_train)
```



Predictions

```
y_pred_rf = rf_model.predict(X_test)
```

Evaluation

```
print("Random Forest Performance:")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Precision:", precision_score(y_test, y_pred_rf))
print("Recall:", recall_score(y_test, y_pred_rf))
print("F1 Score:", f1_score(y_test, y_pred_rf))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_rf))
```

```
Random Forest Performance:
Accuracy: 0.7863733144073811
```

```
Precision: 0.6254295532646048
Recall: 0.48663101604278075
F1 Score: 0.5473684210526316

Confusion Matrix:
[[926 109]
 [192 182]]
```

Random Forest Interpretation

Random Forest improves performance by combining multiple decision trees. It reduces overfitting and usually provides better generalization compared to a single decision tree.

Model Comparison Table

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_log),
        accuracy_score(y_test, y_pred_tree),
        accuracy_score(y_test, y_pred_rf)
    ],
    "Precision": [
        precision_score(y_test, y_pred_log),
        precision_score(y_test, y_pred_tree),
        precision_score(y_test, y_pred_rf)
    ],
    "Recall": [
        recall_score(y_test, y_pred_log),
        recall_score(y_test, y_pred_tree),
        recall_score(y_test, y_pred_rf)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred_log),
        f1_score(y_test, y_pred_tree),
        f1_score(y_test, y_pred_rf)
    ]
})

results
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.805536	0.657233	0.558824	0.604046
1	Decision Tree	0.721079	0.475452	0.491979	0.483574
2	Random Forest	0.786373	0.625430	0.486631	0.547368

Next steps: Generate code with `results` New interactive sheet

Model Comparison

The performance of the three models is compared using accuracy, precision, recall, and F1-score.

- Logistic Regression achieved the highest accuracy and F1-score.
- Decision Tree performed the worst due to overfitting and instability.
- Random Forest improved over Decision Tree but did not outperform Logistic Regression.

Overall, Logistic Regression provides the best balance between precision and recall.

Final Model Selection

Based on the evaluation metrics, Logistic Regression is selected as the final model.

Reasons:

- It achieved the highest accuracy (≈ 80.5%)
- It has the highest F1-score, indicating a good balance between precision and recall
- It generalizes better compared to Decision Tree and Random Forest

Although Random Forest performed well, its recall was lower than Logistic Regression, meaning it missed more churn cases.

Therefore, Logistic Regression is the most reliable model for predicting customer churn in this dataset.

Feature Importance

```
# Get feature names after preprocessing
ohe = log_model.named_steps["preprocessor"].named_transformers_["cat"]
encoded_cat_features = ohe.get_feature_names_out(categorical_cols)

all_features = list(numerical_cols) + list(encoded_cat_features)

# Get coefficients
coefficients = log_model.named_steps["classifier"].coef_[0]

feature_importance = pd.DataFrame({
    "Feature": all_features,
    "Coefficient": coefficients
})

feature_importance = feature_importance.sort_values(by="Coefficient", ascending=False)

feature_importance.head(10)
```

	Feature	Coefficient
16	InternetService_Fiber optic	0.634195
36	Contract_Month-to-month	0.582883
3	TotalCharges	0.536253
35	StreamingMovies_Yes	0.200938
32	StreamingTV_Yes	0.200481
43	PaymentMethod_Electronic check	0.197867
18	OnlineSecurity_No	0.156279
27	TechSupport_No	0.131503
14	MultipleLines_Yes	0.104649
0	SeniorCitizen	0.053449

Next steps: Generate code with `feature_importance` New interactive sheet

Feature Importance Interpretation

The most influential features affecting customer churn include:

- Contract type (month-to-month customers are more likely to churn)